

Báo cáo thực tập: Xây dựng phần mềm Hệ thống Chữ ký số AI-ECDSA

Tích hợp Thị giác Máy tính và Mật mã học Đường cong Elliptic

Sinh viên thực hiện:

Nguyễn Công Trường

Nguyễn Dương Phúc

Hoàng Xuân Đức

Vị trí thực tập: AI Engineer Intern

Đơn vị: SurfaceCity VietNam

Ngành Toán ứng dụng và Tin học
Trường Đại học Khoa Học Tự Nhiên, ĐHQGHN

Hà Nội, Ngày 16 tháng 5 năm 2026

Nội dung báo cáo

- 1 Giới thiệu & Nền tảng Mật mã học
- 2 Dữ liệu & Tiền xử lý
- 3 Phương pháp luận: Thị giác máy tính (Chuyên sâu)
- 4 Quy trình triển khai & AI Copilot
- 5 Kết quả thực tế
- 6 Kết luận & Tầm nhìn tương lai

1.1. Bối cảnh và Đặt vấn đề

Giới hạn của hệ thống cũ

- Các hệ thống ký số hiện tại phụ thuộc hoàn toàn vào thiết bị phần cứng (USB Token) và mã PIN.
- **Lỗi hồng:** Hệ thống chỉ xác thực được *"Ai đang cầm chìa khóa"*, chứ không định danh được *"Ai là người thực sự đang ngồi trước màn hình"*.

Giải pháp đề xuất: Định danh Sinh trắc học (Zero-Trust)

- Ứng dụng **Thị giác máy tính (Computer Vision)** để thay thế hoàn toàn mật khẩu.
- Kết hợp thuật toán mật mã hạng nặng (ECDSA) để đảm bảo tính toàn vẹn và chống chối bỏ của văn bản.

1.2. Cơ sở Mật mã: Hàm Băm & Khóa Bất đối xứng

Để bảo vệ dữ liệu văn bản (PDF, JSON), hệ thống sử dụng kết hợp hai công nghệ mật mã lõi:

- **Hàm băm SHA-256 (Secure Hash Algorithm):**

- Nén toàn bộ file báo cáo thành một chuỗi 256-bit duy nhất.
- Tính chất kháng va chạm: Đổi 1 bit ở file gốc $\Rightarrow$ Chuỗi Hash thay đổi hoàn toàn (Bảo vệ tính toàn vẹn).

- **Mật mã Bất đối xứng (Asymmetric Cryptography):**

- **Private Key (Khóa bí mật):** Chỉ chủ nhân giữ, dùng để ký lên mã Hash.
- **Public Key (Khóa công khai):** Mọi người đều biết, dùng để xác minh chữ ký.

1.3. Thuật toán Ký số Đường cong Elliptic (ECDSA)

Thay vì dùng thuật toán RSA truyền thống, dự án lựa chọn **ECDSA** với đường cong secp256k1 (thuật toán cốt lõi bảo vệ mạng lưới Bitcoin).

Phương trình đường cong Elliptic qua trường hữu hạn p

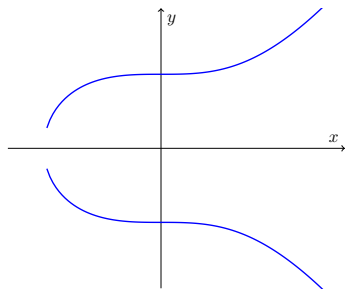
$$y^2 \equiv x^3 + 7 \pmod{p}$$

Tại sao chọn ECDSA?

- Dựa trên bài toán Logarit Rời rạc (ECDLP) cực kỳ khó giải.
- Khóa 256-bit có độ an toàn tương đương khóa RSA 3072-bit.

Ưu điểm cho hệ thống:

- Kích thước tính toán nhỏ gọn.
- Xử lý siêu tốc $\Rightarrow$ Tối ưu RAM khi chạy song song cùng mô hình AI.



Minh họa đường cong $y^2 = x^3 + 7$

2.1. Nguồn dữ liệu thời gian thực (Real-time Stream)

Khác với việc giải mã tĩnh, khóa ECDSA của dự án được bảo vệ bằng luồng nhận diện sinh trắc học.

- **Đầu vào (Input):** Luồng MediaStream từ Webcam của người dùng.
- **Tần số quét:** 30 Frames Per Second (FPS).
- **Nhiều dữ liệu (Noise):** Bức ảnh thường chịu tác động của điều kiện ánh sáng ngược, bóng đổ văn phòng, hoặc hiện tượng nhòe do chuyển động (Motion Blur).

2.2. Tiền xử lý: Dò tìm & Căn chỉnh (Detection & Alignment)

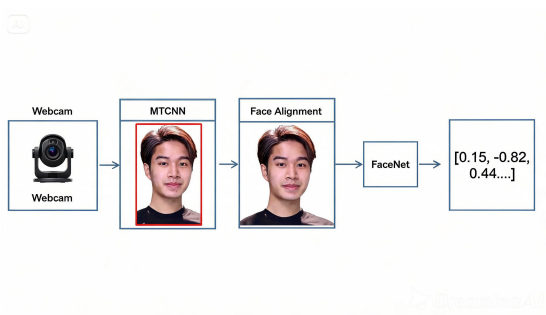
Để mô hình AI đọc được nét mặt, khung hình thô cần trải qua các phép biến đổi hình học:

- ❶ **Face Detection:** Sử dụng mạng MTCNN để khoanh vùng tọa độ khuôn mặt (Bounding Box).
- ❷ **Landmark Detection:** Xác định 68 điểm đặc trưng (mắt, chóp mũi, khóe miệng).
- ❸ **Phép biến đổi Affine (Affine Transformation):**
 - Căn chỉnh dựa trên tọa độ hai mắt để xoay khuôn mặt về phương thẳng đứng.
 - Chuẩn hóa không gian màu (Min-Max Normalization) đưa phân phối pixel về khoảng $[-1, 1]$ để giảm ảnh hưởng của ánh sáng chói.

3.1. Đường ống Xử lý Ảnh (Vision Pipeline)

Để biến luồng video thô thành quyết định bảo mật, hệ thống thực hiện đường ống 4 bước:

- 1 Phát hiện (Detection):**
Định vị Bounding Box bằng MTCNN.
- 2 Căn chỉnh (Alignment):**
Biến đổi Affine dựa trên Landmark.
- 3 Trích xuất (Extraction):**
Bơm qua mạng CNN (FaceNet).
- 4 Đo lường (Matching):**
Tính Cosine Similarity.

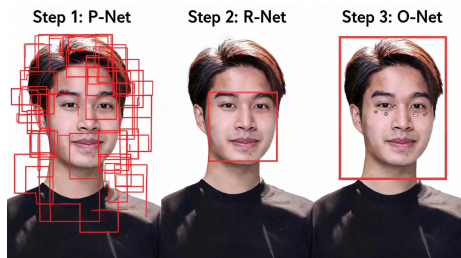


Hình 1: Đường ống xử lý từ Pixel đến Vector

3.2. Mạng MTCNN (Multi-task Cascaded CNN)

Khác với mạng CNN thông thường, MTCNN là một chuỗi 3 mạng lưới xếp tầng (Cascaded) để lọc dần các vùng không phải khuôn mặt.

- **P-Net (Proposal Network):** Quét toàn bộ ảnh, sinh ra hàng nghìn khung hình dự báo (Bounding Boxes) thô.
- **R-Net (Refine Network):** Lọc bỏ các khung hình sai (False Positives).
- **O-Net (Output Network):** Tinh chỉnh khung hình cuối cùng và xuất ra 5 điểm Landmark (Mắt, mũi, khóe miệng).



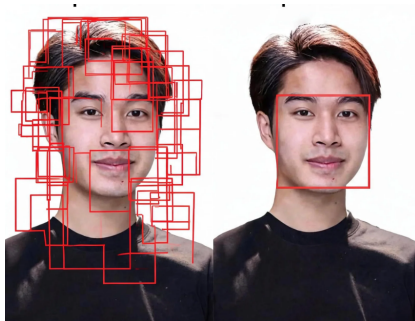
Hình 2: Kiến trúc mạng MTCNN

3.3. Thuật toán Triệt tiêu phi cực đại (NMS)

Khi P-Net quét ảnh, nó sinh ra rất nhiều Box đè lên nhau quanh một khuôn mặt. Nếu đưa tất cả vào tính toán sẽ gây tràn RAM.

Giải pháp NMS

- 1 Tính độ chồng lấp
$$IoU = \frac{Area_Overlap}{Area_Union}$$
- 2 Giữ lại Box có điểm tự tin (Confidence) cao nhất.
- 3 Xóa toàn bộ các Box lân cận có $IoU > 0.5$.



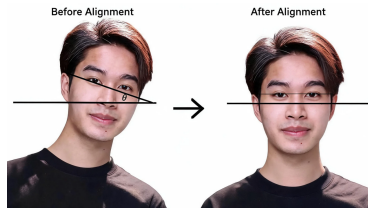
Hình 3: Hiệu quả của thuật toán NMS

3.4. Căn chỉnh khuôn mặt (Face Alignment)

Khuôn mặt chụp từ Webcam thường bị nghiêng, làm giảm độ chính xác của mạng FaceNet.

Phép biến đổi Affine:

- Sử dụng tọa độ 2 mắt (x_1, y_1) và (x_2, y_2) từ O-Net.
- Tính góc nghiêng
$$\theta = \arctan\left(\frac{y_2 - y_1}{x_2 - x_1}\right).$$
- Nhân ma trận điểm ảnh với Ma trận xoay (Rotation Matrix) để dựng thẳng khuôn mặt.



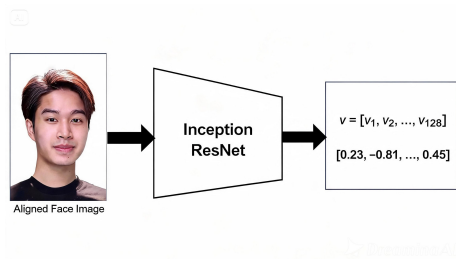
Hình 4:

Phép biến đổi Affine xoay khuôn mặt

3.5. Trích xuất Đặc trưng (Feature Extraction)

Khuôn mặt sau khi căn chỉnh được đưa qua mạng **Inception ResNet v1**.

- **Cơ chế:** Bỏ qua lớp Softmax (Phân loại), chỉ lấy đầu ra của lớp Fully Connected cuối cùng.
- **Kết quả:** Nén toàn bộ cấu trúc sinh trắc học thành một Vector 128 chiều $\vec{v} \in \mathbb{R}^{128}$ nằm trên mặt cầu đơn vị (Hypersphere).

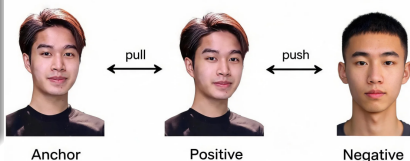


Hình 5: Nén hình ảnh thành Vector đặc trưng

3.6. Hàm mất mát Triplet Loss

Nguyên lý bộ ba (Triplets)

- **A (Anchor):** Ảnh khuôn mặt gốc.
- **P (Positive):** Ảnh khác của **cùng** người đó.
- **N (Negative):** Ảnh của một người **khác**.



Mục tiêu tối ưu hóa:

Kéo véc-tơ của A và P lại gần nhau, đồng thời đẩy A và N ra xa một khoảng lề α .

$$L = \sum \max(\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha, 0)$$

Hình 6: Cơ chế Pull - Push của Triplet Loss

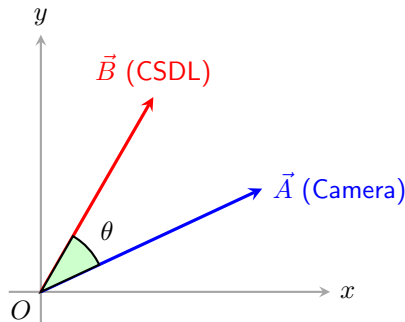
3.7. Đo lường Độ khớp & Ra quyết định

Khi đệ trình, hệ thống so sánh vector thực tế ($\vec{A}$) và vector trong CSDL ($\vec{B}$) bằng Cosine Similarity.

Ra quyết định bảo mật

$$\cos(\theta) = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \|\vec{B}\|}$$

- Góc θ càng nhỏ $\rightarrow \cos(\theta) \rightarrow 1$.
- Mức độ giống nhau tỷ lệ thuận với $\cos(\theta)$.
- Nếu $\cos(\theta) > 0.9$: Kích hoạt ECDSA.
- Nếu $\cos(\theta) < 0.9$: Báo động đỏ (Audit Log).



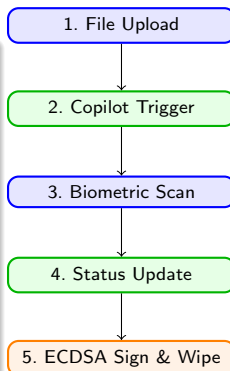
Hình 7: Mô phỏng Cosine Similarity

4.1. Đường ống thực thi toàn hệ thống (System Pipeline)

Sau khi các mô hình AI và Mật mã học được thiết kế độc lập, chúng được ráp nối thành một đường ống khép kín (End-to-End).

Luồng sự kiện (Event Flow)

- ➊ **Input:** User tải file (PDF/JSON) lên Workspace.
- ➋ **UI Bridge:** AI Copilot bắt sự kiện, hiển thị HUD Camera và phát câu thoại hướng dẫn.
- ➌ **Auth Step:** Engine CV chạy ngầm đánh giá định danh sinh trắc học.
- ➍ **Action:** Nếu hợp lệ, Copilot báo thành công → Cấp Private Key → Ký ECDSA.
- ➎ **Cleanup:** Hủy Private Key khỏi bộ nhớ.

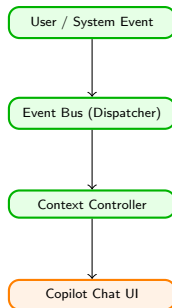


Hình 8: Sơ đồ luồng xử lý toàn hệ thống

4.2. Triển khai AI Copilot: Kiến trúc hướng sự kiện

AI Copilot không phải là một module tĩnh, mà được thiết kế theo mô hình **Event-driven Architecture** để điều phối UI/UX của toàn bộ hệ thống Zero-Trust.

- **Observer Pattern:** Copilot hoạt động như một "người quan sát", liên tục lắng nghe các thay đổi trạng thái từ Data Pipeline và Vision Engine.
- **Quản lý trạng thái (State Management):** Sử dụng React Context API để lưu trữ lịch sử hội thoại và cờ trạng thái (Auth Flags) ở cấp độ toàn cục (Global Scope).
- **Luồng bất đồng bộ (Asynchronous):** Copilot giao tiếp với người dùng trong khi các thuật toán nặng (FaceNet, SHA-256) vẫn chạy ngầm trên Web Workers, đảm bảo UI không bị treo.



Hình 9: Kiến trúc
Event-Driven của Chatbot

4.3. AI Copilot: Xử lý ngữ cảnh (Contextual Logic)

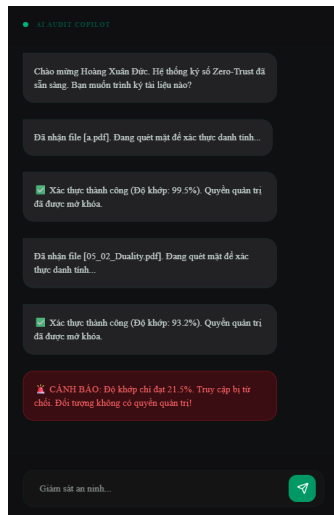
Bộ não của Copilot là một **Máy trạng thái hữu hạn (Finite State Machine - FSM)**. Chatbot sẽ sinh ra các phản hồi tương ứng với từng giai đoạn bảo mật:

Trigger Hệ thống	Node Trạng thái	Hành vi / Phản hồi của Copilot
Người dùng kéo/thả File	FILE_UPLOADED	Mở HUD Camera: " <i>Tệp đã sẵn sàng. Vui lòng nhìn thẳng vào camera.</i> "
Vision Engine đang chạy	SCANNING	Hiển thị Typing (...): " <i>Đang phân tích định danh...</i> "
Cosine Similarity < 0.9	AUTH_FAILED	Cảnh báo: " <i>Phát hiện xâm nhập! Khóa ECDSA bị đóng băng.</i> "
Ký số thành công	SIGN_SUCCESS	Thông báo: " <i>Hoàn tất! File .sig đã được đính kèm an toàn.</i> "

4.4. Triển khai Frontend & Trải nghiệm UX

Để AI Copilot thực sự mang lại cảm giác của một "Trợ lý ảo an ninh", quá trình triển khai Frontend (ReactJS) áp dụng nhiều kỹ thuật tối ưu tương tác:

- **Custom Hooks (useCopilot):** Đóng gói toàn bộ logic hội thoại, giúp mã nguồn tái sử dụng được ở bất kỳ Component nào trên hệ thống.
- **Typing Effect & Animation:** Sử dụng thư viện *Framer Motion* tạo hiệu ứng chữ chạy thời gian thực (như cách ChatGPT phản hồi) giúp giảm cảm giác "chờ đợi máy móc".
- **Auto-Scroll:** Cơ chế tự động cuộn xuống tin nhắn mới nhất.
- **Priority Queue:** Cảnh báo bảo mật màu đỏ luôn được đẩy độ ưu tiên lên cao nhất, kèm hiệu ứng rung (Shake).



5.1. Sản phẩm Phần mềm (Security Portal)

Sản phẩm đầu ra là một Single-Page Application với giao diện Dark Mode "Cyber-Security":

- **Cổng Nhân viên:** Tích hợp HUD Camera, trải nghiệm ký số "Zero-Click" (Không cần nhập mật khẩu hay cắm USB).
- **Manager Dashboard:** Giao diện giám sát theo thời gian thực.
- **Audit Log:** Mọi hành vi sai lệch (người lạ lọt vào camera, tài liệu bị sửa đổi) đều bị chặn đứng và báo động đỏ lên màn hình của Quản trị viên.

5.2. Kết quả Đo lường (Test Cases)

Kịch bản Kiểm thử	Kết quả	Đánh giá Hệ thống
Độ trễ toàn trình	$< 200ms$	Tốc độ sinh trắc học + ECDSA vượt trội so với thao tác cắm USB/nhập PIN vật lý.
Mạo danh định danh	PASS	Người lạ ngồi vào máy $\rightarrow$ Cosine Similarity rớt xuống 28% $\rightarrow$ Khóa ECDSA bị đóng băng.
Tấn công MITM	PASS	Đổi 1 bit trong file $\rightarrow$ Mã Hash thay đổi $\rightarrow$ Chữ ký số báo vô hiệu (Invalid Signature).

6.1. Tổng kết những đóng góp của đề tài

Thành quả cốt lõi

- Hệ thống giải quyết triệt để sự lỏng lẻo của USB Token bằng **Zero-Trust** và **Computer Vision**.
- Tích hợp thành công **FaceNet** và thuật toán **ECDSA** trên môi trường Edge Computing, đảm bảo tốc độ và quyền riêng tư.
- Đem lại trải nghiệm người dùng (UX) hiện đại, tự động hóa cao nhưng vẫn giữ chuẩn an ninh cấp doanh nghiệp.

6.2. Thực trạng mã hóa và "Đám mây đen" Lượng tử

Thuật toán ECDSA hiện tại an toàn tuyệt đối trước máy tính siêu cường cổ điển. (Đó là lý do Satoshi Nakamoto dùng đường cong secp256k1 để bảo vệ mạng lưới Bitcoin trị giá hàng nghìn tỷ đô la).

Mối đe dọa từ Máy tính Lượng tử (Quantum Computing)

- Thuật toán của Shor (Shor's Algorithm) chạy trên máy tính lượng tử có thể giải quyết bài toán Logarit rời rạc (ECDLP) trong **thời gian đa thức**.
- Khi IBM và Google đạt được ngưỡng hàng triệu Qubits ổn định (dự kiến 2030 – 2035), ECDSA và RSA sẽ bị bẻ khóa trong **vài giờ**, đe dọa sụp đổ hệ thống tài chính toàn cầu (bao gồm cả Bitcoin).

6.3. Hướng phát triển Hậu Lượng tử (Post-Quantum)

Để hệ thống Surface City có vòng đời bền vững (Future-Proof) trong kỷ nguyên tới, nhóm đề xuất hướng nâng cấp:

❶ Mật mã Hậu lượng tử (PQC - Post-Quantum Cryptography):

- Chuyển đổi từ thuật toán ECDSA sang các chuẩn mã hóa mới có khả năng kháng lượng tử do NIST công bố (như *Crystals-Dilithium* dựa trên Mạng tinh thể - Lattice-based cryptography).

❷ Liveness Detection: Bổ sung module chống giả mạo sinh thể 3D (bắt buộc người dùng chớp mắt/nhép môi) để chống lại công nghệ Deepfake.

TRÂN TRỌNG CẢM ƠN!

*Cảm ơn Quý Thầy/Cô trường ĐH Khoa học Tự nhiên
và các Anh/Chị tại SurfaceCity đã hỗ trợ em.*

Em xin phép lắng nghe ý kiến phản biện từ Hội đồng.